

: Keras Framework:

high level deep learning Framework

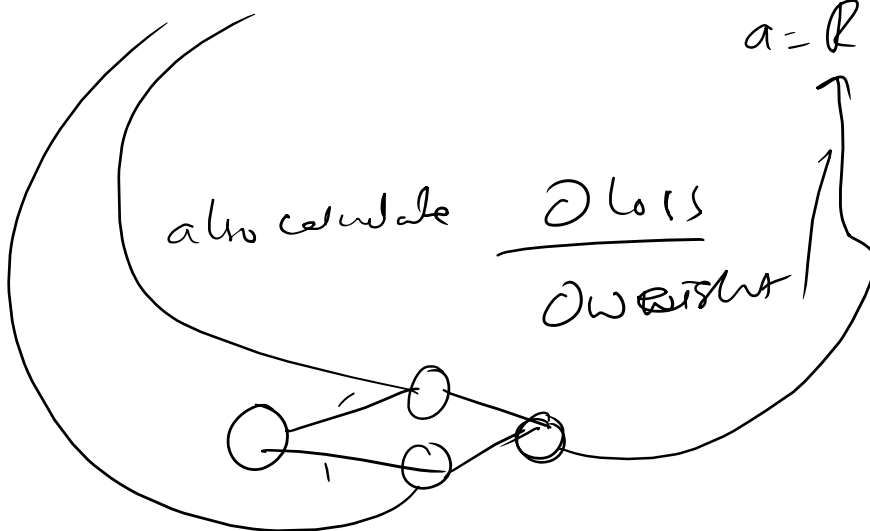
to build and train neural networks

which allow to create models
with building blocks

- ① Dense
- ② Conv2D
- ③ MaxPool2D
- ④ Flatten
- ⑤ Dropout
- ⑥ LSTM

$z = wx + b$ then apply activation

$$a = \text{Relu}(z)$$



Keras (with)

weight initialization

forward propagation

backpropagation

Gradient Calculations

weight updates

CPU

CPU

TPU

→ =

Ready made layers in Keras

Dense

Regular neural network

Conv2D

Image processing — layers

maxpooling2D

Reduce feature size (image)

LSTM

Sequential and Time Series data

embedding

-text processing

Dropout

Reduce overfitting

Batch Normalization

stabilize training

to build neural network we have three API styles

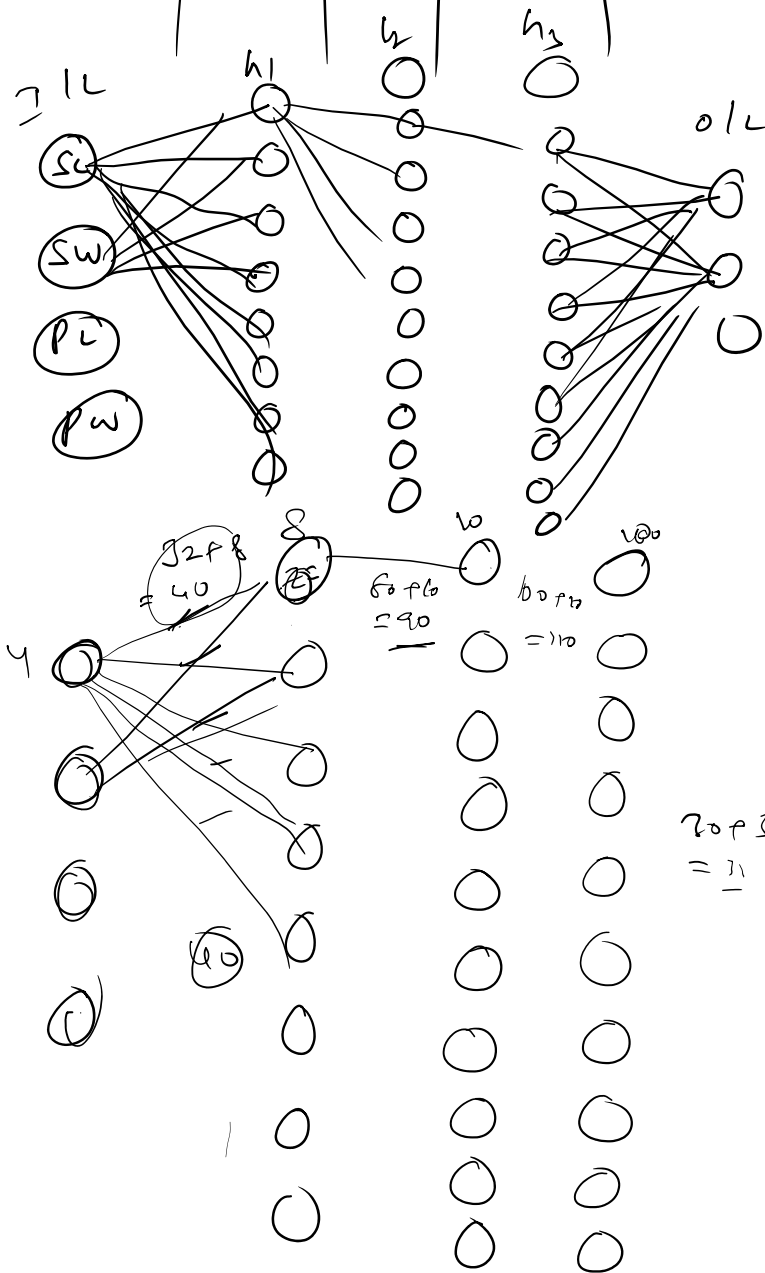
① Sequential API → $\bigcirc // - \bigcirc \bigcirc$

② functional API → $- (-) (-)$

③ model subclassing API → $\boxed{\quad} \boxed{\quad}$

iris data

x				y
SL	SW	PL	PW	species



Sigmoid → binary
 softmax → not binary
MSE linear

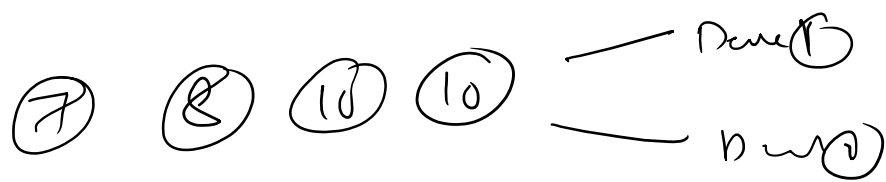
$$\begin{array}{r} 40 \\ 90 \\ 110 \\ \hline 240 \end{array}$$

$$20 + 3 = 23$$

$$x_1, x_2 \rightarrow z = 0.1x_1 + 0.2x_2 + b \rightarrow \sigma(z) = \frac{1}{1 + e^{-z}}$$

functional API!

diabetes = 7 input only



input_layer = Input(shape=(7,))

l1 = Dense(8, activation='relu')(input_layer)

l2 = Dense(10, activation='relu')(l1)

l3 = Dense(10, activation='relu')(l2)

output_1 = Dense(1, activation='sigmoid', name='')
(l3)

output_2 = Dense(1, activation='linear', name='')
(l3)

model = Model(inputs=input_layer, outputs=[output_1, output_2])

Model subclassing:



from keras import layers

```
import keras
```

```
class MyModel(keras.Model):
```

```
    def __init__(self, num_classes=1, h1=8, h2=10,  
                                                           h3=10, *args):
```

```
        self.l1 = layers.Dense(h1, activation='relu')
```

```
        self.l2 = layers.Dense(h2)
```

```
        self.l3 = layers.Dense(h3, activation='relu')
```

```
        self.out = layers.Dense(num_classes, activation='softmax')
```

```
    def call(self, inputs):
```

```
        x = self.l1(inputs)
```

```
        x = self.l2(x)
```

```
        x = self.l3(x)
```

```
        return self.out(x)
```

Sequential API

Dense()

Dense()

Dense()

functional

Dense(1)

Dense(12)

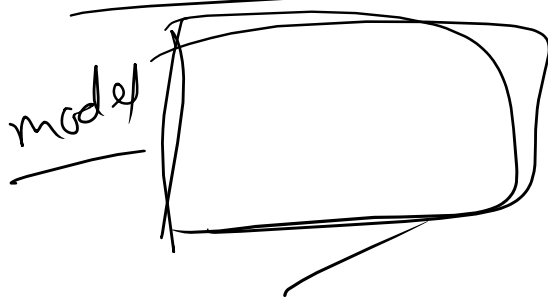
Dense(13)

..

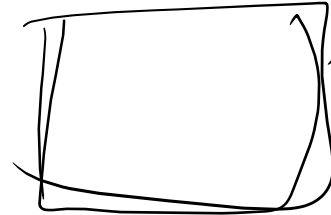
implement

model file

model



deployment



Sequential model → ~~downloaded~~ model

iris-mlp.keras

put it into git repo
✓

app.py — serve:
render.yaml,
requirements.txt
• Python-version → 3.12.11

deploy onto render (aws).